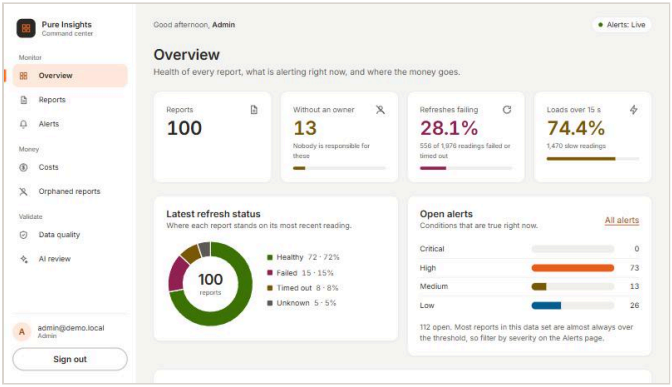
Pure Insights Command Center

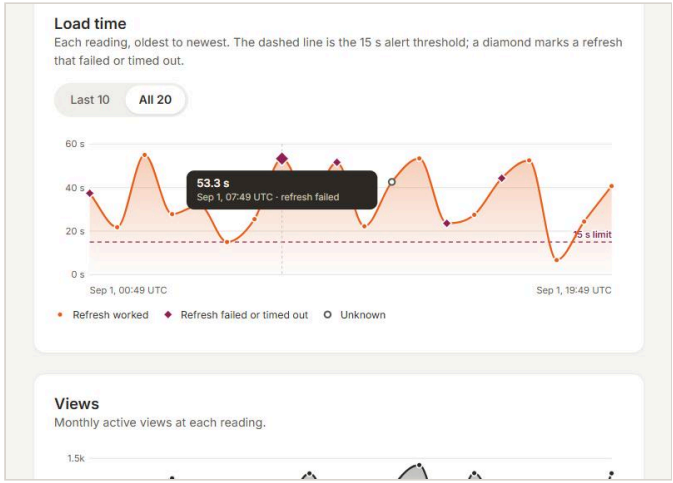
Case Study 3 submission: Technical Data & AI Validation Intern, Monetization (Everpure)

> An admin console that attributes every Snowflake credit to a report, alerts in real time on slow or failing reports, and **proves its own numbers**: a separate Python program re-derives them from the raw CSV and the two are compared item by item.

Code (deliverable 1)	**https://github.com/shivaumsharma/pure-insights-command-center** (React + Node/Express, MIT licence)
REST + WebSocket spec (deliverable 2)	Section 4 below, in full in `docs/api-spec.md`. A test fails if the document and the code's routes ever disagree.
Snowflake SQL suite (deliverable 3)	Section 5 below; the scripts are in `sql/` (`00_schema`, `10` to `14`), explained in `docs/cost-allocation.md`.
Run it (Node 22+)	`npm install` `npm run web:install` `npm run data:build` `npm run web:build` `npm start` → http://localhost:3101. Demo sign-ins (admin, two owners, a viewer, synthetic data only) are in `backend/config/demo-users.json`. `npm run test:all` runs 497 tests; `npm run validate` runs the independent Python check.
Honest status	Done and tested: data cleaning, SQL suite, REST API with three roles, real-time alerts over WebSocket, React dashboard, independent validation, CI green on GitHub. Not done: the SQL has **never been run on a real Snowflake account** (the trial had expired). It runs on a local SQLite stand-in, and it is parsed and name-checked as Snowflake SQL. See section 9.



Overview: fleet health, open alerts, and a map of every report.



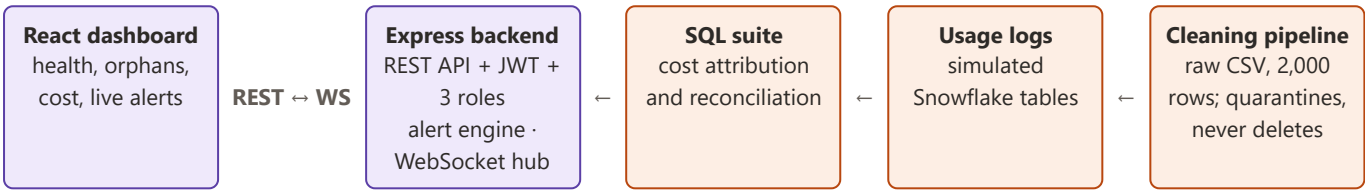
A report's load time with its failed refreshes marked.

1. Where each requirement is met

#	Requirement in the brief	Where, and the evidence
T1	React frontend, Node/Express backend	`frontend/` (Vite, React 19, React Router): sign-in, overview, reports, report detail, alerts, costs, orphans, data quality. `backend/src/` (Express 5). 257 frontend and 240 backend tests.
T1	Role-based access control	Admin / owner / viewer, enforced in the API (`routes/access.js`), not only hidden in the UI. An RBAC matrix test and a scoping test cover every route; 9 mutation checks.
T1	Monitor system health and orphaned reports	`GET /api/health`, `GET /api/orphans`, owner reassignment with an audit trail and undo.
T2	Alert when load time exceeds 15 s, or a refresh fails	`backend/src/alerts/engine.js`. Every one of the 1,181 alert events matches a separate Python implementation.

T2	WebSocket (or SSE) transport	WebSocket at `/ws`, role-filtered, reconnect with replay; 21 tests over real sockets.
T3	SQL over simulated Snowflake `ACCOUNT_USAGE` logs; credits per report; unit cost per active user view	`sql/10_credit_attribution.sql`, `11_unit_cost.sql`; hand-worked fixture tests plus an independent Python re-derivation.
D1-D4	The four defects injected in the 2,000-row CSV	Section 6: orphaned reports, negative credits, latency spikes to 55 s, active-user vs AWS-cost discrepancies. Nothing was deleted; every exception is flagged or set aside with a reason.
R1-R3	The three deliverables	The repository; `docs/api-spec.md`; `sql/`. The full requirement-to-evidence table is `docs/brief-map.md`.

2. Full-stack architecture



- **One process, one address.** The same server serves the API, the WebSocket and the built UI, so there is no CORS to configure. The UI is served with a strict Content-Security-Policy.
- **Authentication.** `POST /api/auth/login` returns a signed JWT (HS256, algorithm pinned, 8 h). Five wrong sign-ins for one email and address in 15 minutes are throttled. Wrong password and unknown email give the same 401.
- **Roles. Admin**: everything, including reconciliation, quarantine, orphans, audit and reassigning owners. **Owner**: health, detail, cost and alerts for the reports they currently own, and a 404 for anyone else's. **Viewer**: fleet and report health, read-only, no cost data of any kind.
- **Snowflake stand-in.** The cost scripts are plain portable SQL. Locally they run unchanged on `node:sqlite`; on Snowflake they run after `USE DATABASE PURE_INSIGHTS;` (`docs/snowflake-run.md`).
- **Decisions are written down.** 46 numbered decisions with the reason and the alternative (`docs/decisions.md`), including every threshold that is my choice rather than the brief's.

3. Real-time alerting

An alert is raised when a report's load time is over **15 s**, or an automated refresh fails or times out. Alerts are computed by an event-time engine as readings arrive and pushed to every connected client the role may see.

Rule	Behaviour
Severity (load time)	over 15 s low · over 25 s medium · over 35 s high · 55 s or more critical. A failed or timed-out refresh is high; three in a row is critical. An `UNKNOWN` status raises nothing and resolves nothing.
Lifecycle	`TRIGGERED` (first time) → `ESCALATED` (worse; sent at once) → `REPEATED` (still true after a 6 h cooldown) → `RESOLVED` (two consecutive clear readings).
De-duplication	74% of readings breach 15 s, so repeats are held back and counted (`suppressed_since_last`): 2,026 breaching readings become 838 alerts needing attention.
Telemetry source	The cleaned observations are replayed at 10 per second through the engine, then loop with timestamps moved forward. Time in alerts is event time, not wall-clock time.
Reconnect	A client sends the last `event_id` and `stream_id` it saw and receives exactly the alerts it missed, once each, then live ones. If events were dropped, or the server restarted, it gets a `gap` message and refetches over REST.

4. REST and WebSocket specification (summary)

Base URL `http://localhost:3101`. JSON everywhere; one error shape: `{ "error": { "code", "message", "details" } }` with 400, 401, 403, 404, 413, 429 and 500 documented. Lists take `limit` (1 to 200) and `offset` and return `{ data, page: { limit, offset, total } }`. Send `Authorization: Bearer <token>`.

Endpoint	Roles	Purpose
`POST /api/auth/login`, `GET /api/auth/me`	public / all	Sign in; who am I
`GET /api/health`	all	Fleet counts, refresh success rate, latency breaches (an owner sees only their reports)
`GET /api/reports`, `/api/reports/:id`	all	List with filters and sort; one report with its recent readings. Cost fields are absent for viewers
`GET /api/alerts`, `/active`, `/summary`	all / admin	Alert history with filters; open conditions worst-first; counts
`GET /api/costs/reports`	admin, owner	Credits, cost per report and per view, idle spend
`GET /api/costs/reconciliation`, `/variance`	admin	The eight reconciliation checks; attributed vs claimed credits per report
`GET /api/costs/aws-mismatch`, `/aws-usage`	admin	Hosting cost that does not fit the views; every reading with its flag (for the chart)
`GET /api/orphans`; `PUT` / `DELETE /api/reports/:id/owner`	admin	Reports with no owner; assign or remove an owner (audited)
`GET /api/quarantine`, `/api/dq-report`, `/api/audit`	admin	Rows set aside with reasons; the data-quality report; owner changes
`GET /api/alerts/:id/triage`, `/api/triage/*`	all / admin	"Explain this alert" (section 8) and its review queue

WebSocket `ws://localhost:3101/ws`

The browser's WebSocket cannot set headers, and a token in the URL leaks into logs, so the token goes in the **first message**.

```
client -> { "type": "auth", "token": "<jwt>", "last_event_id": 41, "stream_id": "9f3a1c2e" }  // last two optional
server -> { "type": "ready", "user": {...}, "last_event_id": 57, "stream_id": "9f3a1c2e", "resumed": true }
server -> { "type": "gap", "reason": "dropped" | "restarted" }   // only if needed; client refetches over REST
server -> { "type": "alert", "event_id": 42, "alert_type": "LATENCY", "lifecycle": "ESCALATED",
            "severity": "high", "report_id": "RPT_201", "value": 41.2, "threshold": 15,
"suppressed_since_last": 2 }
client -> { "type": "ping" }   server -> { "type": "pong" }   server -> { "type": "error", "code", "message" }, then closes
```

Close codes: 4401 not authenticated, bad or expired token, or 5 s auth timeout · 4429 more than 5 connections for a user · 1008 invalid message · 1003 binary frame · 1009 over 4 KB · 1001 server shutting down. The server pings every 30 s. Browser origins other than the server's own (or `ALLOWED_ORIGINS`) are refused with 403. Alerts are filtered by role on the server, so an owner never receives another owner's alerts.

5. Cloud cost SQL: attributing credits to reports

Method. Snowflake bills per warehouse-hour, not per query. The standard way to cost queries on a shared warehouse is to split each warehouse-hour's credits across the queries that ran in it, in proportion to their share of elapsed time. A report's credits are the sum over the hours it ran. Failed and timed-out queries still hold the warehouse, so they keep their share. Credits are conserved: the attributed credits in an hour add up to that hour's metered credits.

```
-- sql/10_credit_attribution.sql (core of it)
SUM(w.elapsed_ms) OVER (PARTITION BY w.warehouse_name, w.window_start)     AS window_elapsed_ms,
CASE WHEN window_elapsed_ms > 0 THEN elapsed_ms * 1.0 / window_elapsed_ms ELSE 0 END AS workload_share,
workload_share * window_credits                                            AS attributed_credits
FROM sim_account_usage.query_history q
JOIN sim_account_usage.warehouse_metering_history m
  ON m.warehouse_name = q.warehouse_name AND q.start_time >= m.start_time AND q.start_time < m.end_time
```

Script	What it does
`00_schema.sql`	Schemas and tables: the simulated `QUERY_HISTORY` and `WAREHOUSE_METERING_HISTORY`, cleaned observations, parameters
`10_credit_attribution.sql`	Splits each warehouse-hour's credits across its queries by elapsed-time share
`11_unit_cost.sql`	Cost per report and **per active view**; idle spend (cost with zero views); orphan status
`12_reconciliation_checks.sql`	Eight PASS / FAIL / INFO checks on the numbers
`13_report_variance.sql`	Attributed vs CSV-claimed credits per report, within 10%
`14_views_vs_aws_cost.sql`	Readings whose AWS hosting cost does not fit their views (3x above or below the fleet's cost per view)

What the checks found on this data

Check	Question	Result
R1	Do attributed credits equal metered credits in every warehouse-hour?	PASS
R2 to R6	Metered credits in hours with no queries; a join silently dropping an observation or a query row; negative credits left after cleaning	PASS (0 each)
R7	Attributed total vs CSV total	INFO: equal, but **true by construction**, so it proves nothing
R8	Per-report attributed credits vs what the CSV claims, within 10%	64 of 100 reports outside tolerance

The R8 result is expected: the supplied data was generated so that each report's credits are independent of its workload. It shows the reconciliation *can* catch a disagreement; it is not evidence that any real report is mis-billed. Dollar figures use a placeholder credit price ($3) and are labelled illustrative in the API (`price_is_placeholder: true`).

6. The four defects in the data, and what happened to each

Defect	Found	Handling
Orphaned reports (`NaN` / `ORPHANED_REPORT`)	242 readings have no usable owner (125 empty, 117 the literal placeholder); 13 reports have none on their latest reading	Flagged, never dropped. The Orphaned reports page lets an admin assign an owner (audited, with undo).
Negative credit values	24 rows, net −161.04 credits	Set aside in a quarantine ledger with a reason code (`/api/quarantine`); not deleted. Cost totals exclude them.
Latency spikes up to 55 s	1,470 of 1,976 readings are over 15 s; 6 sit at exactly 55 s	Alerted by tier; the 55 s ceiling is critical. Repeats are de-duplicated (838 alerts).
Active users vs AWS hosting cost	625 readings (32%) have hosting cost 3x or more away from what their views predict	A chart on the Costs page plots every reading against the fleet's cost per view, with the 3x band; flagged readings are listed. "Active users" are approximated by monthly active views.

Statistical anomaly detection was tested and rejected as the method: every numeric column is spread evenly across its range, so the 1.5x IQR and 3-standard-deviation tests flag nothing, and the 24 negative values sit well inside those fences. They are caught by a rule (credits cannot be negative). Thirteen findings in all, each with evidence, impact, who to route it to and a next step, are in `docs/validation-findings.md`.

7. Quality, and how far to trust the numbers

- **497 automated tests** (240 backend, 257 frontend), each run on a clean machine by GitHub Actions: data-drift check, tests, build, and two end-to-end smoke runs that start the whole product. Tests were **mutation-checked**: deliberately breaking the code in throwaway copies and confirming a test fails. One mutation (removing a divide-by-zero guard) cannot be detected locally and is recorded as such.
- **Independent validation:** `npm run validate` runs a separate Python program that re-derives every headline number from the raw CSV and compares it item by item with the app: 68 checks, all passing. It also parses the SQL as Snowflake SQL, resolves every column and table against the schema, and confirms the CSV column order matches the tables.
- **The spec cannot drift:** a test compares the documented endpoints with the routes actually registered, and calls each on a live server.

Number	Where it comes from	How much to trust it
100 reports, 24 negative rows, 13 unowned	The supplied CSV, re-derived in Python	Solid: counts of real file contents
838 alerts from 2,026 breaching readings	My alert rules; every alert matched by the Python model	The thresholds are my choices; the reduction depends on this data
64 of 100 reports disagree on credits	My simulated usage logs vs the CSV	Shows the check works; says nothing about real billing
Warehouse credits equal CSV credits	Built from the CSV	True by construction; proves nothing

The validator was written by the same author as the app, from the same written rules, so it catches slips and drift, not a wrong reading of the brief.

8. Beyond the brief: "Explain this alert", with the AI checked

Because the role is about validating AI output, each alert can be explained by a model (hosted, or a free local one through Ollama) or by plain rules. The model is not trusted: it receives a numbered evidence pack and must cite which fact supports each claim and the value it read. A checker compares every citation with the real data, tests each cause against a data rule, and rejects numbers not in the evidence. Verdicts: **passed** (shown, with a "how it was checked" list), **could not be verified** (held for an admin to approve or dismiss), or **contradicts the data** (withheld). When the AI's answer is held, everyone else sees the rule-based answer for the same alert if it passed the same checks, labelled as rule-based.

Measured, small sample: four free local models on 10 alerts each passed the checks 6 (Llama 3.1 8B), 3 (Gemma 2 9B), 0 (Qwen 2.5 7B) and 0 (Qwen 2.5 1.5B) times; the rule-based control passed 9 of the same 10. This is a small, untuned sample, not a claim about AI in general, and no hosted model was run.

9. Limits, and what I would do next

- **Never run on real Snowflake.** The trial had expired. The scripts are portable SQL, tested on SQLite with hand-worked fixtures, re-derived in Python, and checked statically as Snowflake SQL. A step-by-step run guide with expected results is in `docs/snowflake-run.md`. The parser is lenient, so a pass means "no typos or malformed structure it can see", not "valid on Snowflake".
- **The usage logs are simulated from the CSV**, so warehouse totals match it by construction. The reconciliation matters on real `ACCOUNT_USAGE` data.
- **The CSV has no timestamps.** Alert order rests on a synthetic timeline (the k-th row of a report is its k-th hourly reading). The columns are statistically unrelated, so rankings reflect the generator and should not be read as business conclusions.

- **Choices that are mine, not the brief's:** the 10% credit tolerance, the 3x hosting band, the severity tiers, the 6 h cooldown, the $3 placeholder price. Each is in the decision log.
- **Next:** run the SQL on a real account and compare; add a flag-only statistical screen beside the rules for real data; add a stored history so alerts survive a restart.

How it was built

The code, tests and docs were written with Claude Code (an AI assistant) at my direction, and the repository says so (`docs/AUTHORSHIP.md`). The code was developed over several sessions before the repository existed, so the commit history groups finished work by area rather than replaying it; every commit carries the date it was really made, and the decision log is the better record of why each choice was made.

Pure Insights Command Center · github.com/shivaumsharma/pure-insights-command-center · Built on synthetic data. Not an official Everpure or Pure Storage product.